

TRACEguard: An Auditability Layer for AI Coding Agents

Research Question

Can a lightweight auditability layer that binds claims to evidence, signals uncertainty, verifies risky outputs, and records high-risk actions make terminal coding agents more transparent, trustworthy, and accountable without removing the speed and accessibility that make them useful?

Hypothesis

If autonomous coding agents are wrapped in a lightweight auditability layer, then they will become more trustworthy and accountable without a major loss of usefulness, because research shows that evidence-grounded outputs, calibrated uncertainty, selective abstention, targeted verification, and action logging improve reliability more effectively than longer explanations alone, and because Claude Code and Codex already expose hooks, approvals, sandboxes, and instruction files that can enforce these controls without retraining the model (Menick et al., 2022; Asai et al., 2024; Dhuliawala et al., 2024; Manakul et al., 2023; OpenAI, 2026a; OpenAI, 2026b; Anthropic, 2026b).

Background

AI coding systems are no longer simple autocomplete tools. Claude Code is an agentic coding tool that can read a codebase, edit files, run commands, and integrate with development tools, while Codex reads AGENTS.md before doing any work and relies on sandboxing, approvals, and hooks to control what the agent can do. As these systems gain file-system and terminal privileges, the main question is no longer just whether they can generate code quickly. It is whether a user can later tell what the agent believed, what evidence it relied on, what it changed, and when it should have asked for permission. Public surveys suggest that this remains an open problem: in the 2025 AI Agent Index, only 10 of 30 deployed agents provided detailed action traces, while 12 of 30 provided no meaningful usage monitoring or only rate-limit notices (Anthropic, 2026a; OpenAI, 2026a; OpenAI, 2026b; Staufer et al., 2026).

This matters because transparency is often confused with verbosity. Human-centered transparency research argues that transparency should help users form accurate mental models rather than simply expose more internals. A 2025 Nature Machine Intelligence study found that users overestimated the accuracy of LLM answers when those answers came with default explanations, and that longer explanations increased user confidence even when the extra length did not improve answer accuracy. A classroom study reached a similar conclusion:

showing a system prompt or goal summary did not significantly improve performance, while simple reliability disclaimers were more effective at reducing overreliance when advice was wrong. These findings suggest that better trust will not come from making coding agents “talk more.” It will come from making them easier to verify and easier to audit (Liao & Vaughan, 2024; Steyvers et al., 2025; Bo et al., 2024).

Key Findings

Finding: Strong language models are still unreliable enough that trust cannot be based on fluency alone. Evidence: In TruthfulQA, which contains 817 questions across 38 categories, the best model in the original study was truthful only 58% of the time, compared with 94% for humans. In HaluEval, ChatGPT generated unverifiable hallucinated content in about 19.5% of responses. In FActScore, ChatGPT achieved only 58% factual precision on long-form biography generation. What it means: A coding agent that sounds certain is not automatically a trustworthy agent. Unsupported factual claims about libraries, commands, environment behavior, or external knowledge should be treated as potentially wrong unless they are grounded and checked (Lin et al., 2022; Li et al., 2023; Min et al., 2023).

Finding: Evidence binding and adaptive retrieval are among the strongest low-friction ways to improve trustworthiness. Evidence: GopherCite produced high-quality supported answers 80% of the time on a Natural Questions subset and 67% on ELI5; when it abstained on the third of questions it was least sure about, performance rose to 90% and 80%. Self-RAG then showed that selective retrieval with self-reflection can improve both factuality and verifiability: its 13B model reached 80.2 FActScore on biography generation and 70.3 citation precision with 71.3 citation recall on ASQA. CRAG strengthens this pattern further by evaluating retrieval quality before using it: its lightweight retrieval evaluator reached 84.3% accuracy on PopQA retrieval judgments, far above ChatGPT’s 58.0%. What it means: TRACEguard should require important factual claims to be linked either to repository evidence, retrieved external evidence, or an explicit “unsupported/tentative” label. Retrieval should be used adaptively, not on every turn (Menick et al., 2022; Asai et al., 2024; Yan et al., 2024).

Finding: Selective verification and calibrated abstention outperform generic “explain your reasoning” prompts. Evidence: Chain-of-Verification improved MultiSpanQA F1 by 23%, from 0.39 to 0.48, and improved FActScore by 28%, from 55.9 to 71.4. On list-style factual tasks, it more than doubled precision from 0.17 to 0.36 while reducing hallucinated negatives from 2.95 to 0.68. The same paper found that instruction-tuning and chain-of-thought did not reliably reduce hallucinations. SelfCheckGPT showed that even black-box models can be checked after the fact: its NLI variant reached 66.08 AUC-PR for factual-sentence detection and 74.14 Pearson correlation at passage level, while the stronger prompt-based version reached 67.09 AUC-PR and 78.32 Pearson correlation. Separate work on verbalized confidence found that the best

prompt combinations can produce confidence scores with an average deviation of about 7% from empirical accuracy, while AbstentionBench showed that abstention remains unsolved and that reasoning fine-tuning can worsen abstention by 24% on average. What it means: TRACEguard should use a lightweight confidence field on every answer and trigger a challenge pass only when risk is high, instead of forcing a second pass on every output (Dhuliawala et al., 2024; Manakul et al., 2023; Yang et al., 2025; Kirichenko et al., 2025).

Finding: Accountability in coding agents requires runtime controls and receipts, not just better prompts. Evidence: A recent preprint on coding agents found that permissive frameworks, including Claude Code and Codex CLI, showed overeager out-of-scope action rates ranging from 5.4% to 27.7%, while an ask-to-continue framework was much lower, at 0.2% to 4.5%. In paired scenarios on Claude Code, stripping a consent declaration increased the overeager rate from 0.0% to 17.1%. Meanwhile, the AI Agent Index found that transparency, monitoring, and guardrails vary widely across deployed agents. Official documentation also shows that Codex uses sandbox and approval layers, while Claude Code provides deterministic hooks and permission rules for controlling tool use. What it means: A trustworthy coding agent needs both pre-action control and post-action traceability. Prompting alone cannot guarantee either one (Qu et al., 2026; Staufer et al., 2026; OpenAI, 2026b; OpenAI, 2026c; Anthropic, 2026b; Anthropic, 2026c).

Insight / Proposed Idea

Open-source developers and platform teams building on Claude Code and Codex should adopt TRACEguard, a lightweight default auditability layer for terminal coding agents. TRACEguard would not attempt to expose hidden chain-of-thought. Instead, it would make agent behavior auditable through five concrete rules: Tool-boundary gating, Reference binding, Abstain-or-ask thresholds, Challenge-pass verification, and Event logging. This proposal is more defensible than “show the reasoning,” because the literature shows that longer explanations can inflate confidence, while evidence, uncertainty, verification, and traces are easier for users to inspect and challenge (Steyvers et al., 2025; Liao & Vaughan, 2024; Bo et al., 2024).

Tool-boundary gating means the agent stays autonomous for normal repository-local work, but must pause for approval when it tries to cross a clear risk boundary such as network access, secrets access, writes outside the workspace, or destructive shell commands. This adds almost no model token cost because it is mainly enforced through existing permission and approval systems rather than extra prompting. That trade-off is worth it because recent evidence suggests that permissive coding-agent workflows are especially vulnerable to out-of-scope actions (OpenAI, 2026b; Anthropic, 2026c; Qu et al., 2026).

Reference binding means important factual claims must be tagged as one of three types: supported by repository evidence, supported by retrieved external evidence, or unsupported and therefore tentative. Abstain-or-ask thresholds mean the model must emit a short confidence field and refuse to sound certain when evidence is weak. These two rules add only a small amount of output overhead, but they directly address the two biggest trust failures in the literature: unsupported factual claims and overconfident language. They also preserve accessibility because they do not require retraining or expensive always-on multi-agent workflows (Menick et al., 2022; Asai et al., 2024; Yan et al., 2024; Yang et al., 2025; Kirichenko et al., 2025).

Challenge-pass verification means TRACEguard runs a second verification step only when needed, for example when confidence is low, when the answer contains unsupported external claims, or when the agent is about to produce a risky final recommendation. This selective design matters because the literature supports verification, but it also shows that constant extra reasoning is not the same as constant extra accuracy. By making verification conditional rather than universal, TRACEguard keeps most of the speed and autonomy of current coding agents while importing the strongest factuality gains from CoVe- and SelfCheckGPT-style checking (Dhuliawala et al., 2024; Manakul et al., 2023).

Event logging means every session ends with a compact audit receipt: files touched, commands run, approvals requested, evidence sources used, tests executed, unresolved uncertainties, and any policy violations. This is the most important part of the proposal because it turns transparency into accountability. Users do not need a model to merely sound responsible; they need a record they can inspect after the run. Claude Code and Codex already support the main implementation hooks needed for this design through permissions, hooks, and instruction files, which means TRACEguard is realistic as a wrapper rather than a whole new model. That keeps it widely accessible to students, hobbyists, and small developer teams (Staufe et al., 2026; OpenAI, 2026a; OpenAI, 2026c; Anthropic, 2026b; Anthropic, 2026d).

To make the protocol practical, we implemented a prototype CLI that installs TRACEguard instructions, generates audit receipts, and configures agent-specific transparency rules for Claude Code and Codex. The prototype is available at: github.com/jaysflight1/traceguard.

Conclusion

This paper asked whether a lightweight auditability layer could make coding agents more transparent, trustworthy, and accountable without removing their usefulness. The evidence suggests that the answer is yes, but only if transparency is defined as something users can verify. Existing research shows that fluent explanations alone can inflate trust, while evidence binding, calibrated uncertainty, selective abstention, targeted verification, and action logging

offer stronger support for reliability. Because Claude Code and Codex already expose practical control points such as approvals, hooks, and instruction files, TRACEguard serves as a low-cost wrapper to replace expensive model retraining.

References

- Anthropic. (2026a). Claude Code overview. Claude Code Docs.
<https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>
- Anthropic. (2026b). Automate workflows with hooks. Claude Code Docs.
<https://docs.anthropic.com/en/docs/claude-code/hooks-guide>
- Anthropic. (2026c). Claude Code settings. Claude Code Docs.
<https://docs.anthropic.com/en/docs/claude-code/settings>
- Anthropic. (2026d). Hooks reference. Claude Code Docs.
<https://docs.anthropic.com/en/docs/claude-code/hooks>
- Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. ICLR. <https://arxiv.org/abs/2310.11511>
- Bo, J. Y., Kim, S., Kross, S., et al. (2024). Disclosures & disclaimers: Investigating transparency and reliability mechanisms for LLM tutors. HCOMP.
<https://www.cs.toronto.edu/~ashton/pubs/llm-disclose-hcomp2024.pdf>
- Dhuliawala, S., Komeili, M., Xu, J., et al. (2024). Chain-of-Verification reduces hallucination in large language models. Findings of ACL.
<https://aclanthology.org/2024.findings-acl.212.pdf>
- Kirichenko, P., Ibrahim, M., Chaudhuri, K., & Bell, S. J. (2025). AbstentionBench: Reasoning LLMs fail on unanswerable questions. arXiv. <https://arxiv.org/abs/2506.09038>
- Li, J., Cheng, X., Zhao, W. X., Nie, J.-Y., & Wen, J.-R. (2023). HaluEval: A large-scale hallucination evaluation benchmark for large language models. EMNLP.
<https://aclanthology.org/2023.emnlp-main.397/>
- Liao, Q. V., & Vaughan, J. W. (2024). AI transparency in the age of LLMs: A human-centered research roadmap. Harvard Data Science Review.
<https://hdsr.mitpress.mit.edu/pub/aelql9qy>
- Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA: Measuring how models mimic human falsehoods. ACL. <https://aclanthology.org/2022.acl-long.229/>
- Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. EMNLP.
<https://aclanthology.org/2023.emnlp-main.557.pdf>

- Menick, J., Trebacz, M., Mikulik, V., et al. (2022). Teaching language models to support answers with verified quotes. arXiv. <https://arxiv.org/abs/2203.11147>
- Min, S., Krishna, K., Lyu, X., et al. (2023). FActScore: Fine-grained atomic evaluation of factual precision in long-form text generation. EMNLP. <https://aclanthology.org/2023.emnlp-main.741/>
- OpenAI. (2026a). Custom instructions with AGENTS.md. Codex documentation. <https://developers.openai.com/codex/guides/agents-md>
- OpenAI. (2026b). Agent approvals & security. Codex documentation. <https://developers.openai.com/codex/agent-approvals-security>
- OpenAI. (2026c). Hooks. Codex documentation. <https://developers.openai.com/codex/hooks>
- Qu, Y., Zhang, Y., Zhang, Y., et al. (2026). Overeager coding agents: Measuring out-of-scope actions on benign tasks. arXiv. <https://arxiv.org/abs/2605.18583>
- Staufer, L., Feng, K., Wei, K., et al. (2026). The 2025 AI Agent Index: Documenting technical and safety features of deployed agentic AI systems. FAcCT. <https://aiagentindex.mit.edu/data/2025-AI-Agent-Index.pdf>
- Steyvers, M., Tejeda, H., Kumar, A., et al. (2025). What large language models know and what people think they know. Nature Machine Intelligence. <https://www.nature.com/articles/s42256-024-00976-7>
- Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective retrieval augmented generation. arXiv / ICLR submission. <https://arxiv.org/abs/2401.15884>
- Yang, D., et al. (2025). On verbalized confidence scores for LLMs. OpenReview. <https://openreview.net/pdf?id=CVRdNQvFPE>